UNIVERSITY OF MINNESOTA
Driven to Discover®

What We Didn't Cover

CSCI 5980: Game Engine Architecture

Evan Suma Rosenberg | CSCI 5980 | Spring 2026

Semester Summary

We developed a minimal-yet-functional game engine with the following features:

- A node-based scene graph hierarchy emphasizing composition over inheritance
- An extensible framework for registering engine services
- A platform independence module that encupsulates window creation and management
- A shader-based rendering module using modern OpenGL
- A resource module for asynchronous file loading and resource management
- Procedural generation of common mesh primitives

Semester Summary (Continued)

- An offline asset conditioner that processes meshes, textures, and audio clips
- A physics module that provides collision testing and rigid body simulation
- Collision and rendering support for height-mapped terrain
- A first-person camera that supports kinematic user control and collision testing
- An audio module that supports 3D spatialization
- Various other ad hoc features when needed

Some Engine Statistics

- 84 source files total
  - 35 .cpp files
  - 49 header files
  - 6,682 total physical lines
  - 5,991 code lines

What We Didn’t Cover

- Game engines are extremely large, complicated systems
- A production-ready game engine will take a team years to develop
- There is way more content than can be covered in a single course!
- The textbook is ~1200 pages long
- We probably covered less than half of it

Human Interface Devices

Human Interface Devices

- Games are interactive simulations, so the player needs a way to provide input
- A wide range of HIDs are used for gaming
  - Joypads, keyboards, mice, track balls, motion controllers
  - Specialized devices: steering wheels, dance pads, electric guitars
- Some HIDs also provide **outputs** as feedback to the player
- Game engines must read, process, and utilize these inputs

Interfacing with a HID

- Polling: explicitly query device state each frame
  - Example: XInput's `XInputGetState()` for Xbox controllers
- Interrupts: device signals the CPU only when state changes
  - Common for mice and keyboards
- Wireless devices: Bluetooth controllers communicate via protocol
  - Often handled on a separate thread
  - Then exposed like a traditional polled device

Types of Inputs

- Digital buttons: pressed or not pressed (a single bit)
- Analog axes: a range of values (thumb sticks, triggers)
- Relative axes: deltas from last read (mice, track balls, mouse wheels)
- Accelerometers: measure acceleration on the Wiimote and DualShock
- 3D orientation: estimated from accelerometers using gravity
- Cameras: Wiimote IR sensor, PlayStation Eye, Microsoft Kinect

Types of Outputs

- Rumble: vibrations produced by unbalanced motors in the controller
- Force feedback: motors that resist the player's motion
  - Common in arcade driving games with steering wheels
- Audio: speakers in the Wiimote, headphone jacks on modern controllers
  - Often used for VoIP in multiplayer games
- Other: LEDs, light bars, memory card slots, specialized device outputs

Engine HID System Features

- Dead zones
- Analog signal filtering
- Chord and gesture detection
- Managing multiple HIDs for multiple players
- Cross-platform HID abstraction
- Controller input remapping
- Context-sensitive controls
- The ability to disable specific inputs

Tools for Debugging and Development

Logging and Tracing

- `printf` / `cout` debugging is still useful, especially in real-time code
  - Some bugs only appear at full speed or after long event sequences
- Verbosity levels: control which messages print at runtime
- Channels: categorize output by subsystem (animation, physics, AI, etc.)
- Mirror all output to log files for after-the-fact debugging
- Crash reports: dump player state, stack trace, memory state

Debug Drawing

- API for drawing colored lines, shapes, and 3D text in the game world
- Common primitives: lines, spheres, points, axes, bounding boxes, text
- Drawn in world space or screen space, with or without depth testing
- Each primitive has a **lifetime** so it persists between draw calls
- Invaluable for visualizing math: trajectories, sight lines, blast radii

Debug Drawing

Figure 10.1. Visualizing the line of sight from an NPC to the player in *The Last of Us: Remastered* (© 2014/™ SIE. Created and developed by Naughty Dog, PlayStation 4).

Debug Drawing

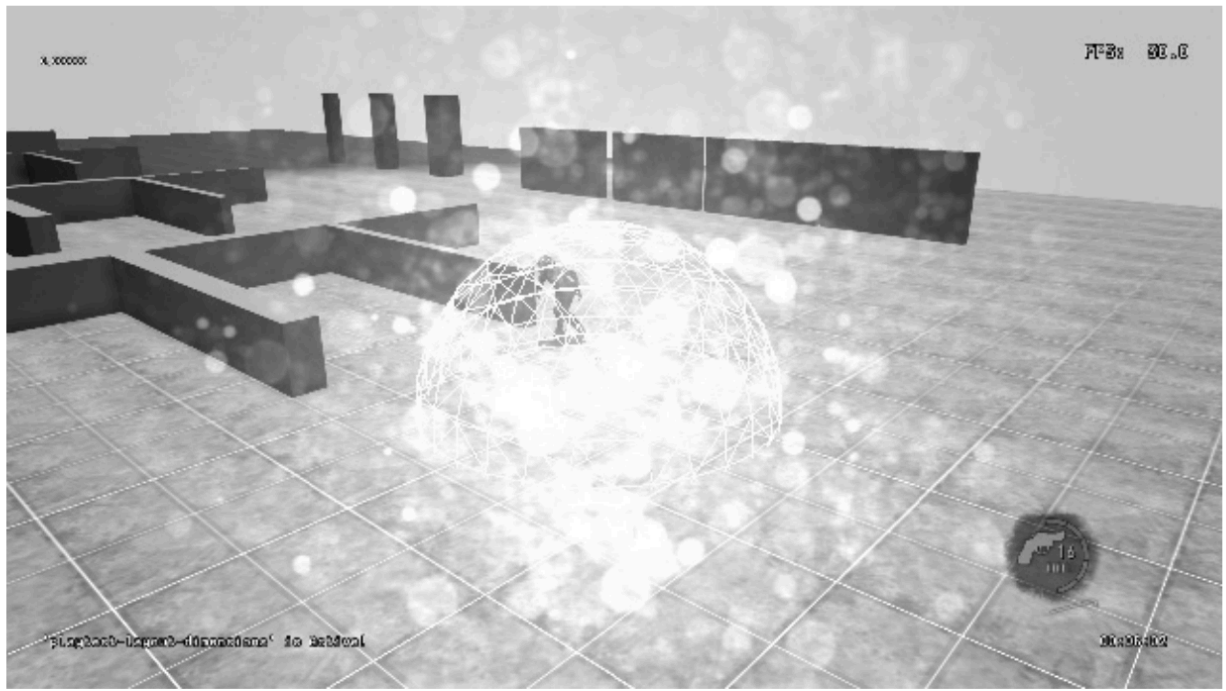

Figure 10.2. Visualizing the expanding blast sphere of an explosion in the Naughty Dog engine.

Debug Drawing

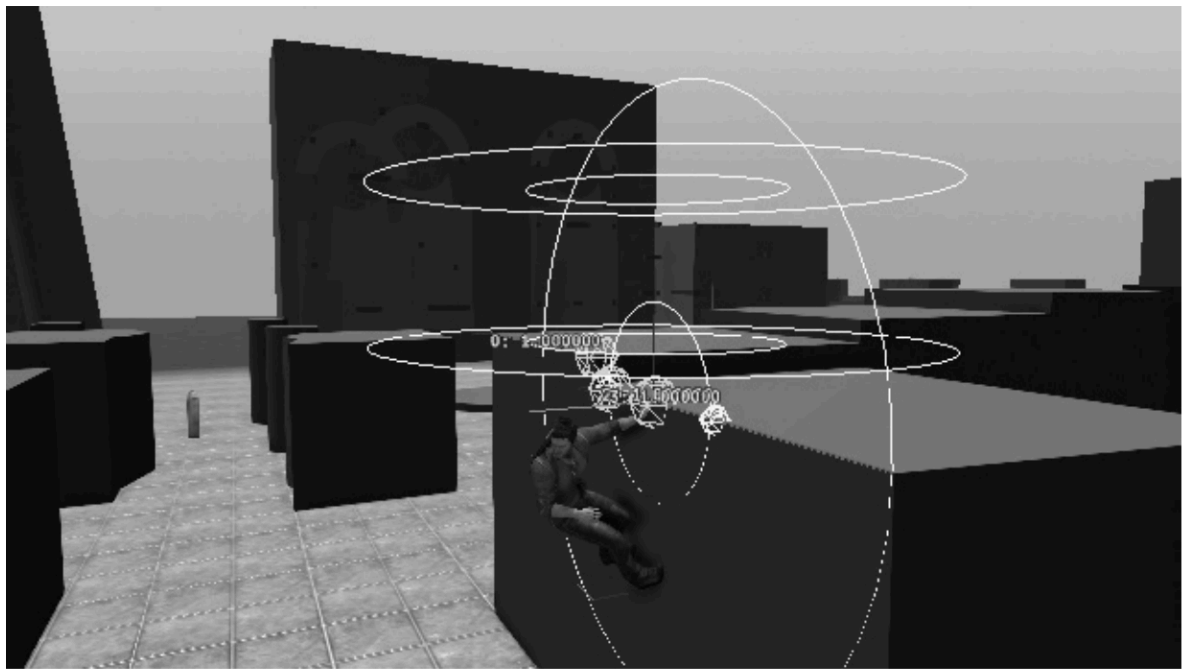

Figure 10.3. Spheres and vectors used in Drake's ledge hang and shimmy system in the *Uncharted* series (© 2014/™ SIE. Created and developed by Naughty Dog, PlayStation 3).

Debug Menus and Consoles

- Adjust engine settings at runtime without recompiling
- Menus: hierarchical, navigable with the joypad
  - Toggle booleans, adjust values, call functions
- Consoles: command-line interface
  - Less convenient than menus, but more powerful
  - Especially flexible when tied to the engine's scripting language
- Bringing up the menu typically pauses the game

Debug Menus

Figure 10.5. Main development menu in *The Last of Us: Remastered* (© 2014/™ SIE. Created and developed by Naughty Dog, PlayStation 4).

Debug Menus

Figure 10.8. Background meshes turned off (*The Last of Us: Remastered* © 2014/™ SIE. Created and developed by Naughty Dog, PlayStation 4).

Debug Consoles

Figure 10.9. The in-game console in *Minecraft*, overlaid on top of the main game screen and displaying a list of valid commands.

Debug Cameras, Pausing, and Cheats

- **Debug camera**: detach from the player and fly anywhere in the world
- **Pause, single-step, slow motion, fast motion**
  - Decouple the gameplay clock from the real-time clock
- **Cheats**: break the rules to speed up development and testing
  - Invincible player, give weapon, infinite ammo, fly with collisions off

Screenshots and Movie Capture

- Capture the frame buffer to disk in standard image formats
- Options:
  - include or exclude debug primitives
  - include or exclude HUD
  - set resolution
- Some engines support full movie capture
- PS4 continuously records the last 15 minutes for post-crash analysis

In-Game Profiling

- Measures where the frame's time is being spent
- Programmer annotates code blocks; the profiler times them
- Display as raw cycles, microseconds, or percentage of the frame
- Hierarchical profiling: tracks inclusive vs exclusive time
- Track call counts to distinguish slow functions from frequently-called ones
- Some engines export to CSV for analysis in Excel

Debug Consoles

Figure 10.11. The timeline mode in *Uncharted: The Lost Legacy* (© 2017/™ SIE. Created and developed by Naughty Dog, PlayStation 4) shows exactly when various operations are performed across a single frame on the PS4's seven CPU cores.

Memory Stats and Leak Detection

- Memory is constrained on consoles and "min spec" PCs
- Tracking is harder than wrapping `malloc` and `free`
  - Third-party libraries may bypass your allocator
  - Main RAM and video RAM are managed differently
  - Engines often use multiple specialized allocators
- Displayed as tables or graphical bar charts
- Helpful failure modes for missing assets
  - Red text where a missing model would be
  - Pink texture for a missing texture

Memory Stats and Leak Detection

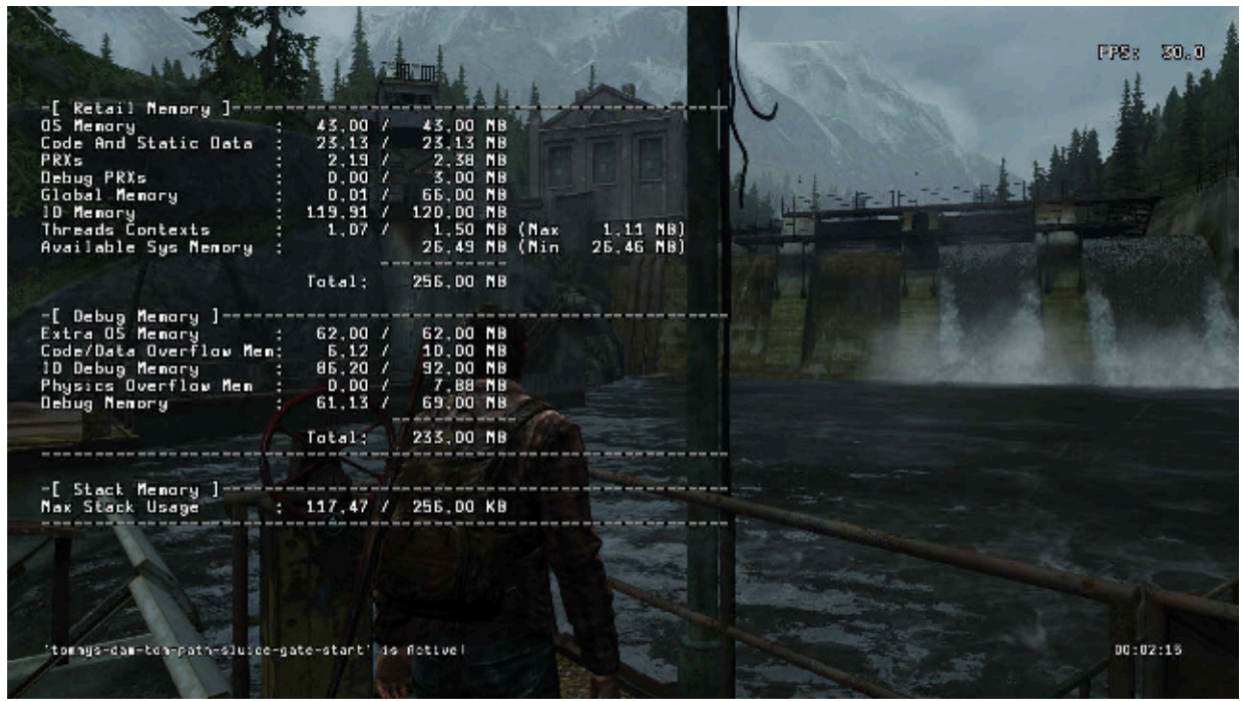

Figure 10.14. Tabular memory statistics from Naughty Dog's engine.

Animation Systems

Types of Character Animation

- Cel animation: 2D sprite-based, used in early games
- Rigid hierarchical: a tree of rigid pieces
  - Works for robots and machinery
  - Suffers from "cracking" at the joints on fleshy characters
- Per-vertex and morph targets: animate vertices directly
  - Morph targets often used for facial animation
- Skinned animation: most prevalent technique today
  - A skin mesh is bound to a hidden skeleton
  - Vertices can be weighted to multiple joints

Skeletons

- A hierarchy of rigid pieces called joints
  - The terms "joint" and "bone" are used interchangeably
- Each joint has an index from 0 to N-1
  - Children always appear after their parent in the array
- Each joint typically stores:
  - A name (string or hashed id)
  - Its parent’s index
  - An inverse bind pose transform

Poses

- **Bind pose**: the reference pose, often a T-pose
  - The pose of the mesh before being bound to the skeleton
- **Local pose**: a joint's pose relative to its parent
  - Stored as SRT (scale, quaternion rotation, translation)
  - Almost all blending is done on local poses
- **Global pose**: a joint's pose in model space
  - Computed by walking the hierarchy and concatenating local poses

Animation Clips

- A clip causes the character to perform a single well-defined action
  - Walk cycle, run cycle, throw, fall, etc.
- Each clip has a local timeline with a duration T
  - Time is continuous, sampled at 30 or 60 samples per second
- Up to 10 channels per joint (S, Q, T components)
- Metachannels carry events, locator transforms, or material parameters
- Many animation clips and meshes can target a single skeleton

Skinning

- Each vertex is bound to one or more joints, each with a weight
  - Four joints per vertex is a typical limit
- A **skinning matrix** transforms a vertex from bind pose to current pose
  - Combines the inverse bind pose with the joint's current global pose
- The renderer receives a **matrix palette**, one matrix per joint
- Vertices bound to multiple joints use a weighted average of skinning results

Animation Blending

- Combines two or more poses to produce an output pose
- LERP blending: linear interpolation between local poses
  - Used for cross-fades, directional locomotion, 1D and 2D blends
- Additive blending: applies a difference clip on top of a target
  - Useful for stance variation, aim, look-at, locomotion noise
- Partial-skeleton blending: per-joint blend weights via a blend mask

The Animation Pipeline

- Clip decompression and pose extraction
- Pose blending across all active clips
- Global pose generation
- Post-processing (IK, rag dolls, procedural adjustments)
- Recalculation of global poses if needed
- Matrix palette generation for the renderer

Scripting

What is Scripting?

- A scripting language lets users control and customize the behavior of an application
- In a game engine, script provides high-level access to engine features
- Used by programmers and non-programmers to build new games or "mod" existing ones
- Data-definition languages: populate data structures consumed by the engine
  - Often declarative, parsed offline or at load time
- Runtime scripting languages: execute within the engine at runtime
  - Used to extend or customize hard-coded engine functionality
  - The focus of this discussion

Characteristics of Game Scripting Languages

- Interpreted: byte code executed by a virtual machine (VM)
  - Easier to embed and reload than a native DLL
- Lightweight: small VM, modest memory footprint
- Rapid iteration: changes can often be reloaded without restarting the game
- Convenient: custom syntax for common tasks
  - Events, FSMs, timing, tweakable parameters, etc.

Common Game Scripting Languages

- QuakeC: Quake's custom C-like scripting language; helped birth the mod community
- UnrealScript: Unreal's custom object-oriented language (no longer supported in UE4)
- Lua: small, fast, embeddable; used in World of Warcraft and many games
- Python: clean syntax, reflective, object-oriented; used in Panda3D
- Pawn (formerly Small): lightweight, dynamically typed, with built-in FSM support
- C#: compiled rather than interpreted, supported by Unity and Godot

Scripting Architectures

- **Scripted callbacks**: hook functions for customizing engine behavior
- **Scripted event handlers**: respond to game or engine events in script
- **Extending object types**: derive or attach script classes to native objects
- **Scripted components**: implement components or properties in script
- **Script-driven engine or game**: script runs the show; native code is the library

Native Language Interface

- The VM is embedded in the engine and runs script code on demand
- Two-way communication is typical
  - Engine calls script functions or methods
  - Script calls into native code through a function lookup table
- Example:
  - Naughty Dog’s DC scripts call C++ functions registered in a global table

A Brief List of Other Topics

Graphics and Rendering Topics

- Physically based rendering
- Visual effects and overlays
- Advanced lighting and global illumination
- Deep learning super sampling (DLSS)
- And plenty more...

Other Topics

- Game world editors
- World chunk data formats
- Loading and streaming game worlds
- Saving and loading game state
- Concurrency and job systems
- Events and message passing
- Networked multiplayer
- And plenty more...

Thoughts about Software Development and AI

Thoughts about Software Development and AI

- Software engineering is in the midst of massive disruptive change
- Coding agents are evolving on a weekly basis
  - Claude Code just got frontend design tool
  - Codex just added computer use (for Mac only)
- Open-source agents like OpenClaw and Hermes are being used for personal assistants and other automated tasks
- It's basically the wild west for agentic coding workflows
  - AGENTS.md/CLAUDE.md, skills, plugins, MCP servers, etc.
- Your carefully designed, elegant workflow might suddenly stop working whenever a new software/model update gets released

Challenges for Teaching and Learning

- To learn how to use agentic coding tools effectively, you need to:
  - Experiment with different workflows
  - Read about what other people are doing with them
  - Build a bunch of stuff, save the prompts and skills that work for re-use

- Most CS professors don’t have a lot of time to code
  - Many of them don’t program at all anymore
  - Many only know about coding agents from reading news articles
  - Many have only a vague understanding of what SOTA coding agents can do now

Challenges for Teaching and Learning

- Throughout your career, you will need to be responsible for your learning
- This is even more important with the prevalance of coding agents!
- It's now very easy to write tons of code with zero understanding of how it works
- That might be fine for one-off programs, but not for serious software
- Production software has other concerns beyond just "getting it working"
  - Long-term maintainability, robustness against security exploits, etc.
- How can you learn how to develop software when an agent is writing the code?

Vibe Coders: Three Archetypes

- Non-programmers:
  - Just wants to build something, not interested in the code
  - Excited because coding agents allow them to create things they couldn't before
  - Mostly a net positive, although there are risks with using AI naively

- Junior developers:
  - Understands how to code, but still learning how to be a software engineer
  - Over-reliance on coding agents inhibits their learning
  - Abstaining entirely from coding agents inhibits their productivity
  - This is the group with the biggest risk factor!

Third Archetype: Senior Developers

- Consider a senior developer experienced in designing and developing complex software
- Can learn a new programming language or SDK relatively easily
- Works with the agent to design the software architecture
  - Puts a lot of effort into this phase
- Offload the tedious part (code generation) to the agent
  - Keeps implementation tasks well-defined and narrowly scoped
- Reviews the code afterward, discusses with the agent if something doesn't make sense
  - May have the agent refactor code to resemble something they would write
- Coding agents are a force amplifier for their capabilities

Questions?